

Project nr 10

Group 32

by Adam Shahumi and Santiago Puig-Terrero López

1. Introduction

With the exponential growth of e-commerce, customer reviews have become a critical metric for evaluating product quality and consumer satisfaction. The aim of this project is to develop a machine learning model capable of predicting a numerical review score based solely on the textual content of a review. We frame this task as a multi-class classification problem, where the input is a raw text string and the target output is a discrete ordinal rating ranging from 1.0 to 5.0 stars. We utilize a subset of the Amazon Books Reviews dataset to train and evaluate our models.

A primary challenge in utilizing real-world review data is the inherent class imbalance; online reviews are typically heavily skewed toward positive (5-star) ratings. If left unaddressed, models often exploit this imbalance by disproportionately predicting the majority class, artificially inflating overall accuracy while failing to learn the semantic nuances of minority classes (e.g., 2-star or 3-star reviews). To counteract this, we propose utilizing a perfectly balanced subset of the data to ensure equal representation across all five classes.

To solve this classification problem, we implement and evaluate three distinct algorithmic approaches: a linear baseline (Logistic Regression), a tree-based ensemble (Random Forest), and a neural architecture (Multi-Layer Perceptron). Furthermore, we conduct an ablation study to analyze the impact of removing standard English stop-words on the model's contextual understanding.

2. Methodology

To process the unstructured text data and train our classifiers, we developed a sequential pipeline consisting of text preprocessing, feature extraction, and model training.

2.1 Text Preprocessing and Feature Extraction

Raw textual data cannot be directly interpreted by standard classification algorithms. We establish a standard cleaning function that converts text to lowercase and strips non-alphabetic characters. Following text cleaning, we utilize Term Frequency-Inverse Document Frequency (TF-IDF) vectorization to map the text into a high-dimensional mathematical space. Unlike a simple word-count approach (Bag of Words), TF-IDF scales the frequency of a word in a specific review by its rarity across the entire dataset, naturally highlighting terms that are highly descriptive of sentiment. The removal of common English stop-words is controlled via a boolean flag to facilitate our ablation study.

2.2 Classification Algorithms

We selected three distinct machine learning paradigms to implement and compare, allowing us to evaluate how different mathematical architectures handle high-dimensional text data:

- **Logistic Regression (Linear Baseline):** Models the probability of a specific class by evaluating a weighted linear combination of input features. It is highly interpretable, fast to train, and often performs surprisingly well on sparse text data by assigning proportional positive or negative weights to sentiment-bearing words.
- **Random Forest (Tree-Based Ensemble):** Trains a multitude of independent decision trees on random subsets of the data. Because decision trees make non-linear splits, this model can capture complex interactions between words that a linear model might omit. However, tree-based models can sometimes struggle with the extreme sparsity of TF-IDF vectors.
- **Multi-Layer Perceptron (Neural Network):** An artificial neural network designed to capture complex, non-linear relationships between word pairings. An MLP processes TF-IDF vectors through hidden layers of neurons, allowing the model to learn deep, hierarchical semantic representations before executing a final classification.

2.3 Evaluation Metrics

To rigorously evaluate model performance and navigate the systemic challenges imposed by the "Accuracy Paradox" discovered in Phase 1, we utilize four complementary multi-class classification metrics. Because our final experimental framework is executed on a perfectly balanced 5-class distribution (where each class c in $\{1.0, 2.0, 3.0, 4.0, 5.0\}$ represents exactly 20% of the corpus), these metrics provide an un-skewed assessment of lexical boundaries

without being manipulated by dominant class majorities. The metrics are mathematically defined below:

- **Classification Accuracy:** Measures the global proportion of correctly predicted review scores out of the total evaluation instances (N). While misleading on imbalanced datasets, it serves as a robust benchmark on our balanced 10,000-review evaluation set.

$$\text{Accuracy} = \frac{\sum_{i=1}^N \mathbb{I}(\hat{y}_i = y_i)}{N}$$

Where $\hat{y}_i$ is the predicted rating, y_i is the true rating, and $\mathbb{I}$ is the indicator function.

- **Precision (Per-Class):** Quantifies the model's exactness by measuring the ratio of correctly predicted positive instances for class c to the total instances predicted as class c . High precision minimizes false positives (e.g., misclassifying a critical 1-star review as a highly positive 5-star review).

$$\text{Precision}_c = \frac{TP_c}{TP_c + FP_c}$$

- **Recall (Per-Class):** Quantifies the model's completeness by measuring the ratio of correctly predicted positive instances for class c to all actual ground-truth members of class c . Maximizing recall ensures that the pipeline successfully captures minority sentiment scores rather than ignoring them in favor of neighboring boundaries.

$$\text{Recall}_c = \frac{TP_c}{TP_c + FN_c}$$

(Where TP_c , FP_c , TN_c , and FN_c represent the True Positives, False Positives, True Negatives, and False Negatives for a given target rating class c , respectively.)

- **Macro-Averaged F1-Score:** Computes the harmonic mean of Precision and Recall, calculated independently for each class and averaged uniformly across all five star-ratings. This serves as our primary structural health metric because it penalizes models that fail to generalize on intermediate, structurally ambiguous classes (such as 2-star and 3-star categories).

$$F1_c = 2 \times \frac{\text{Precision}_c \times \text{Recall}_c}{\text{Precision}_c + \text{Recall}_c}, \quad \text{Macro-F1} = \frac{1}{5} \sum_{c=1}^5 F1_c$$

A sample output would look as such:

```
--- Training Logistic Regression ---
Logistic Regression Accuracy: 0.4533
```

	precision	recall	f1-score	support
1.0	0.56	0.55	0.56	2000
2.0	0.41	0.40	0.41	2000
3.0	0.38	0.36	0.37	2000
4.0	0.38	0.38	0.38	2000
5.0	0.54	0.57	0.55	2000
accuracy			0.45	10000
macro avg	0.45	0.45	0.45	10000
weighted avg	0.45	0.45	0.45	10000

3. Experiments and Progressive Optimization

The development of our final solution was conducted iteratively across seven distinct experimental phases. We progressively modulated our data pipeline and algorithmic hyperparameters, evaluating the impact of each modification to combat bias, manage model capacity, apply regularization, and map data saturation patterns.

3.1 Phase 1: The Baseline and the "Accuracy Paradox"

Our initial baseline experiment utilized a randomly extracted subset of 50,000 reviews with untweaked algorithms and a 5,000-word vocabulary limit. This baseline achieved a deceptively high overall accuracy of 66.11% using the Multi-Layer Perceptron.

However, examining the classification report revealed a severe class imbalance: over 60% of the test data consisted of 5.0-star reviews. The models artificially inflated their accuracy by predominantly guessing the majority class, resulting in an abysmal recall for minority classes (e.g., Logistic Regression achieved a mere 0.12 recall for 2.0-star reviews).

To correct this bias, we constructed a perfectly balanced dataset containing exactly 10,000 reviews for each of the five rating classes (total training pool of 40,000 reviews, with 10,000 held out for a fixed test set). As expected, due to the "Accuracy Paradox," the overall test accuracy dropped to an average of ~47%. However, achieving ~47% on a balanced 5-class problem is a massive statistical lift above random chance (20%), and the recall for intermediate classes (2.0, 3.0, and 4.0) improved dramatically—in some cases by over 200%. This verified that the models were finally learning true semantic nuances rather than exploiting statistical imbalances. This balanced dataset was used for all subsequent phases.

--- Training MLP (Neural Network) ---					
MLP (Neural Network) Accuracy: 0.6611					
	precision	recall	f1-score	support	
1.0	0.52	0.51	0.52	638	
2.0	0.46	0.04	0.07	497	
3.0	0.39	0.19	0.26	818	
4.0	0.42	0.24	0.30	1964	
5.0	0.72	0.93	0.81	6083	
accuracy			0.66	10000	
macro avg	0.50	0.38	0.39	10000	
weighted avg	0.61	0.66	0.61	10000	

--- Training MLP (Neural Network) ---					
MLP (Neural Network) Accuracy: 0.4750					
	precision	recall	f1-score	support	
1.0	0.58	0.56	0.57	2000	
2.0	0.41	0.44	0.43	2000	
3.0	0.41	0.33	0.37	2000	
4.0	0.41	0.41	0.41	2000	
5.0	0.54	0.63	0.58	2000	
accuracy			0.47	10000	
macro avg	0.47	0.47	0.47	10000	
weighted avg	0.47	0.47	0.47	10000	

3.2 Phase 2: Expanding Model Capacity and Vocabulary

To improve upon the ~47% balanced baseline, we attempted to expand the predictive capacity of our models. We deepened the MLP to two hidden layers, reduced regularization in Logistic Regression, increased the Random Forest to 300 estimators, and expanded the TF-IDF vocabulary to 10,000 words.

- **Impact:** The added capacity benefited the tree-based ensemble, pushing Random Forest accuracy from 43.86% to 45.41%. Conversely, the expanded vocabulary introduced extreme "noise" (rare words and typos) that caused the linear and neural models to slightly overfit, leading to minor drops in their test accuracies.

3.3 Phase 3: Testing Feature Availability

To isolate the cause of the overfitting observed in Phase 2, we conducted a stress-test experiment: we maintained the highly complex model architectures but restricted the TF-IDF vocabulary back to 5,000 words.

- **Impact:** This configuration proved to be the absolute "sweet spot" for Random Forest, yielding its peak accuracy of 45.57%. The restricted, less-noisy feature space allowed the 300 trees to make highly generalized splits. However, this setup caused severe overfitting in the MLP and Logistic Regression models. Stripped of the larger vocabulary, these high-capacity algorithms memorized the training data by assigning excessive weight to common terms, dropping their test accuracies to 46.68% and 44.91%, respectively.

3.4 Phase 4: Regularization and Initial Optimization

Realizing that aggressive models overfit to constrained feature spaces, our final optimization focused on intelligent regularization and noise filtering rather than raw capacity. We applied strict document frequency thresholds ($\text{min_df}=5$, $\text{max_df}=0.85$) to the vectorizer to eliminate both hyper-rare typos and ubiquitous uninformative terms. Furthermore, we applied an L1 penalty (Lasso) to the Logistic Regression model to act as an automatic feature selector, and increased the L2 regularization ($\alpha=0.05$) in the MLP.

- **Impact:** These tweaks successfully stabilized the algorithms. The models recovered substantially from the previous overfitting, with Logistic Regression climbing to 46.05% and the MLP reaching 46.93%. By penalizing mathematical complexity and providing a cleaner feature space, we achieved our most robust, generalized baseline model states.

3.5 Phase 5: Ablation Study on Syntactic Context (Stop-Words)

Throughout our iterative testing, we conducted an ablation study to evaluate the core NLP assumption that removing standard English stop-words improves text classification. To ensure a rigorous comparison in our optimized run, we dynamically selected our highest-performing baseline architecture—the Multi-Layer Perceptron—to undergo the ablation test.

The ablation study demonstrated that retaining stop-words slightly degraded performance or introduced redundant dimensions that increased convergence time without providing supplementary statistical power. Stripping stop-words allowed the model to isolate pure sentiment indicators.

3.6 Phase 6: Automated Hyperparameter Grid Search (GridSearchCV)

To fully maximize the potential of our pipelines, we migrated our configurations into an automated GridSearchCV framework across a 3-fold cross-validation scheme to find the definitive optimal boundaries for the model.

For the TF-IDF pipeline, the global maximum performance was established by pairing an n-gram expanded vocabulary with a strict frequency filtering limit ($\text{max_df}=0.5$) and a shallow, single-layer MLP architecture (100,). This configuration yielded the project's global peak performance on unseen data: **47.16% accuracy**.

In parallel, we evaluated the limitations of frequency-based vectorization by implementing a pre-trained contextual Transformer model (all-MiniLM-L6-v2) to generate 384-dimensional dense sentence embeddings. When optimizing the model configurations for these dense contextual embeddings, we discovered severe architectural mismatches. For instance, transitioning from

an L1 penalty to an L2 penalty was mandatory for Logistic Regression; while L1 successfully performed feature selection on the sparse 5,000-dimensional TF-IDF matrix by forcing noisy weights to zero, applying it to BERT's dense 384-dimensional vectors resulted in destructive information loss. L2 regularization smoothly penalized large weights without eliminating features, preserving the continuous semantic space.

The automated grid search determined that the dense BERT vectors required a much deeper, two-layer architecture consisting of hidden layers of size (256, 128), utilizing a ReLU activation and an aggressive weight penalty ($\alpha=0.1$) to successfully decode the complex non-linear semantic spaces. This optimized BERT-MLP pipeline achieved a final test accuracy of **42.86%**.

Model Pipeline Configuration	Hyperparameters Found	Fixed Test Accuracy
Optimized TF-IDF + MLP	Vocabulary:5000, ngram_range=(1,2), max_df=0.5, Network: (100,), alpha=0.1	47.16%
Optimized BERT + MLP	Embeddings: 384-dim Dense, Network: (256,128), Activation: relu, alpha=0.1	42.86%

3.7 Phase 7: Data Saturation and Learning Curve Analysis

To assess the data efficiency and operational boundaries of the optimized TF-IDF + MLP pipeline, a progressive Learning Curve Analysis was conducted. Holding an unseen evaluation set of 10,000 reviews completely static, the training pool was systematically scaled down from 40,000 samples to 5,000 samples in steps of 5,000.

The empirical results demonstrated a classic logarithmic data saturation curve. Between 40,000 and 25,000 samples, the model displayed diminishing returns; truncating the training set by 37.5% incurred a marginal performance penalty of only 1.20% accuracy (falling from 47.16% to 45.96%). This indicates that by 25,000 instances, the n-gram expanded TF-IDF vocabulary has sufficiently converged on the core semantic indicators of the dataset's multi-class star ratings.

Conversely, a severe performance degradation cliff was observed when the training pool fell below 15,000 instances. At the lowest macro-boundary of 5,000 samples, predictive accuracy collapsed to 42.70%.

To isolate the absolute mathematical floor of the pipeline, a secondary high-resolution experiment was executed, zooming directly into micro-dataset configurations. The training data was scaled down from 5,000 samples to 500 samples in granular decrements of 500 instances.

The empirical results mapped out a stark performance collapse. While the architecture maintained a reasonable degree of predictive integrity down to 2,000 samples (achieving 36.89% accuracy), it encountered a catastrophic performance cliff immediately thereafter. Between 2,000 and 1,500 training instances, predictive accuracy collapsed by an abrupt 6.73% absolute percentage points, landing at 30.16%. At the final tested boundary of 500 training samples, the model's accuracy deteriorated to 24.16%. Given that the random-guess baseline for this 5-class distribution is exactly 20.00%, a model trained on 500 samples operates at a state near statistical uselessness.

Macro & Micro Learning Curve Trend:

LEARNING CURVE ANALYSIS SUMMARY	
Training Samples	Fixed Test Accuracy
40,000	47.1600%
35,000	46.7300%
30,000	46.6600%
25,000	45.9600%
20,000	46.0000%
15,000	45.2300%
10,000	44.8300%
5,000	42.7000%
4,500	41.5700%
4,000	41.4000%
3,500	39.6500%
3,000	40.1600%
2,500	38.1000%
2,000	36.8900%
1,500	30.1600%
1,000	30.7000%
500	24.1600%
[SYSTEM] Execution complete. Log permanently saved.	

4. Discussion

The empirical findings across our seven experimental phases reveal several critical insights regarding feature engineering, model capacity, and data requirements for multi-class ordinal sentiment text classification tasks.

4.1 The Sparse vs. Dense Paradigm Mismatch

The most striking result of this study is that a localized, custom-tailored TF-IDF vectorizer (47.16% accuracy) outperformed a state-of-the-art pre-trained BERT Transformer architecture (42.86% accuracy). While counterintuitive given modern deep learning trends, this outcome underscores the difference between static semantic embeddings and task-specific vocabulary customization.

The optimized TF-IDF vectorizer was explicitly built from scratch on the review corpus, leveraging unigrams and bigrams while applying a strict frequency ceiling ($\text{max_df}=0.5$). This allowed the model to ignore ubiquitous background words and heavily penalize terms that did not differentiate between classes, resulting in highly informative features for identifying explicit sentiment shifts (e.g., distinguishing a 1-star review from a 2-star review via strong modifying bigrams like "not recommended").

In contrast, the pre-trained BERT model generated general-purpose sentence embeddings. Because these embeddings were "frozen" and not fine-tuned end-to-end on our specific classification task, the downstream MLP classifier struggled to decode general semantic similarity into precise ordinal star boundaries. Furthermore, our experiments proved that dense continuous vectors require fundamentally different regularization and structural setups than sparse matrices. Applying feature-selective penalties like L1 regularization to dense spaces actively results in destructive information loss, whereas shallow architectures are sufficient for sparse spaces but deeper, non-linear hidden layers are mandatory to unpack dense embeddings.

4.2 Matrix Sparsity and the Data Cliff

The learning curve analysis successfully mapped out the mathematical transition from data saturation to data starvation. The collapse of accuracy from 36.89% at 2,000 samples to 24.16% at 500 samples highlights a fundamental structural limitation of pairing high-dimensional feature spaces with minimal sample sizes.

Mathematically, when the dimensionality of the feature space ($D = 5,000$) vastly exceeds the number of available training instances ($N \leq 1,500$), the resulting feature matrix becomes

underdetermined and sparse. The Multi-Layer Perceptron loses its ability to calculate stable mathematical boundaries across the network weights. Instead of extracting generalizable text representations, the neurons are forced to memorize localized lexical noise unique to those few hundred training instances, resulting in overfitting and a total failure to generalize on unseen test data. This defines an absolute operational threshold for our system: a minimum of 2,000 instances is structurally required to keep the model viable, while 25,000 instances represents the economic optimal sweet spot before reaching a plateau of diminishing returns.

5. Conclusions

This project successfully designed, optimized, and analyzed an end-to-end machine learning pipeline for multi-class review score prediction using the Amazon Books Reviews dataset. By addressing the "Accuracy Paradox" through strict class balancing, we shifted our models from exploiting statistical majorities to genuinely learning semantic and sentiment-bearing text structures.

Our research yields three definitive conclusions:

1. **Vocabulary Control Outperforms Static Deep Learning:** For dedicated text classification tasks, a carefully regularized, localized n-gram TF-IDF pipeline with strict frequency thresholds can outperform out-of-the-box static embeddings from deep learning Transformers by isolating task-specific lexical signals more cleanly.
2. **Architectural Co-dependency:** High-capacity classification models like Multi-Layer Perceptrons are highly sensitive to their input representations. Sparse, high-dimensional matrices are best processed via shallow, regularized networks, whereas dense, low-dimensional contextual spaces require deeper network capacities paired with non-destructive smooth regularization (L2) to prevent information loss.
3. **Logarithmic Data Dependencies:** The pipeline exhibits clear data requirements bounded by matrix dimensionality. The model achieves functional stability at 2,000 samples, reaches its optimal operational efficiency at 25,000 samples, and saturates completely at 40,000 samples, offering a clear guide for balancing computational costs against predictive returns in real-world deployments.

Future work will focus on the end-to-end fine-tuning of the Transformer layers directly on the review text corpus to allow the dense embeddings to dynamically adapt to the nuances of ordinal star ratings, potentially overcoming the current limitations of static contextual vectors.